Regularization 之 Dropout

A Beginner's Note on Dropout, Generalization and Training Tricks

Dropout • Regularization • Train/Eval

1 为什么深度学习需要 Regularization？

核心想法：当模型容量很强、参数很多、训练时间很久，而数据又有限时，模型不仅会学习到样本中的稳定规律，还会把噪声、偶然模式、局部细节一起记住。训练集表现看起来很好，但验证集和测试集效果下降，这就是过拟合。

出现过拟合的本质：模型把训练集学得太“细”了

Regularization 的目标不是单纯“让训练误差变大”，而是通过控制模型有效复杂度，使模型在未见数据上表现更稳定，也就是提升 generalization。

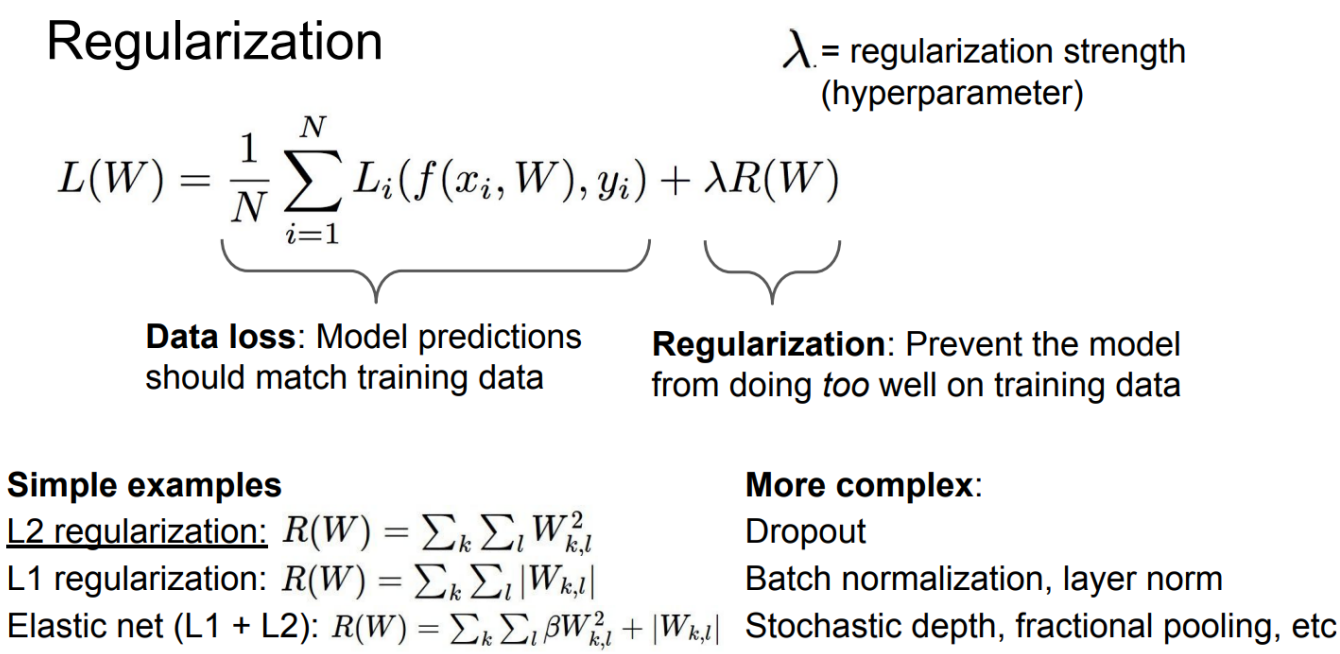

图 1: Regularization 示意图

统一视角：很多 regularization 方法都可以理解为：抑制模型对训练集细节和噪声的过度拟合，从而提升泛化能力。对于显式参数惩罚类方法，常写成

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i; W), y_i) + \lambda R(W),$$

其中前一项是 data loss，后一项是 regularization term，λ 用来控制 regularization strength。

需要注意的是，并不是所有 regularization 都直接写成这种“损失函数 + 惩罚项”的形式。像 Dropout 更像是在训练过程中向网络注入随机扰动，从而间接限制模型对固定激活路径的过度依赖。

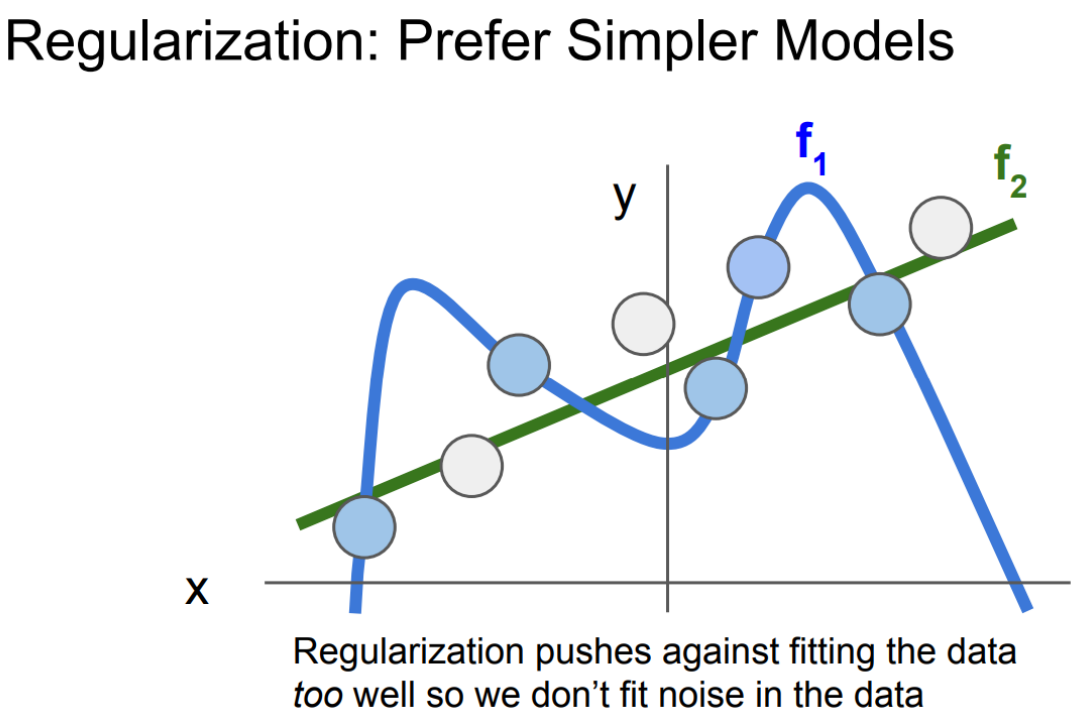

图 2: 可视化 Regularization 作用

可以把它理解成四条主线：从广义角度看，很多提升泛化能力的方法都可以放到 regularization 的讨论框架下：

- 显式参数惩罚：L2 regularization、L1 regularization、Elastic Net
- 随机扰动类方法：Dropout、Stochastic Depth、Fractional Pooling
- 归一化类技巧：Batch Normalization、Layer Normalization
- 控制训练过程：Early Stopping

其中需要注意：像 L1/L2 这类方法是显式 regularization；而 BatchNorm、LayerNorm 的主要作用是稳定训练与优化过程，但在实践中也常常带来一定泛化收益。

2 Dropout

为什么它能抑制过拟合

核心想法：每次前向传播时，Dropout 都会随机屏蔽一部分单元，相当于在共享参数的条件下对大量稀疏子网络做近似平均。它的直观作用是打破神经元之间过强的 co-adaptation，避免模型过度依赖某条固定激活路径，从而缓解过拟合、提升泛化能力。

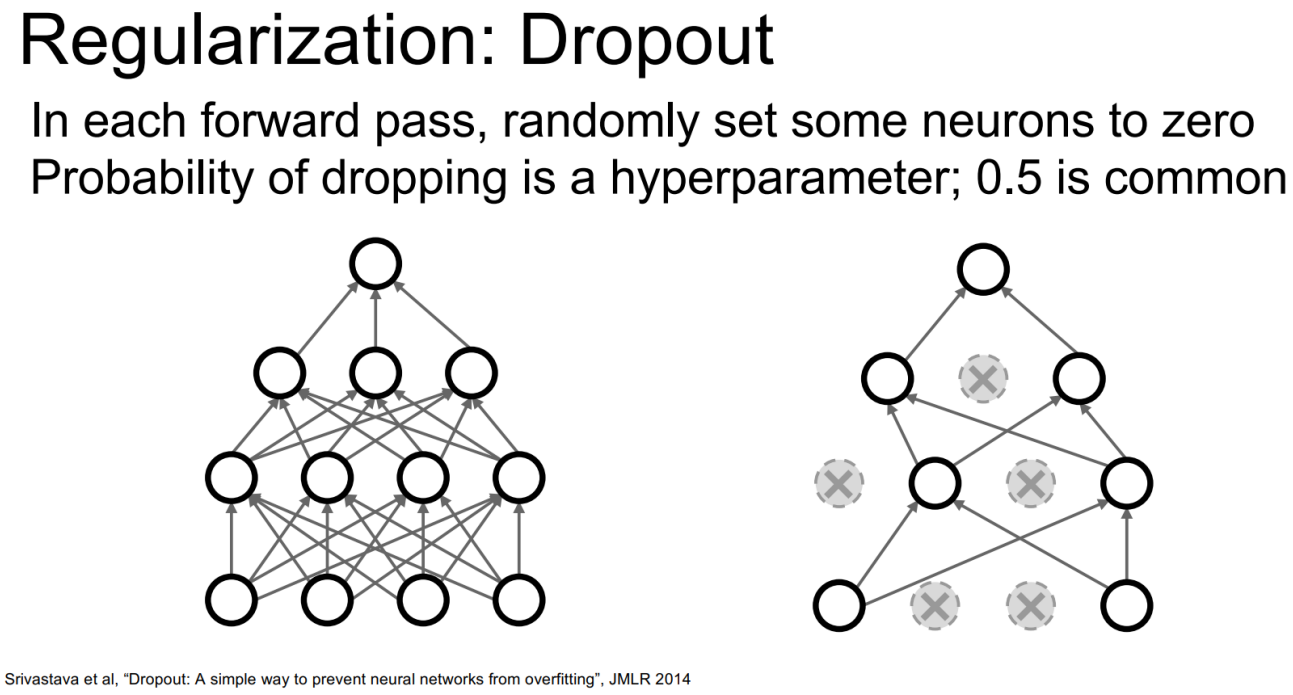

图 3: Dropout 的直观示意图

训练阶段做了什么：给定隐藏层激活向量 h，丢弃率为 p，掩码 m 逐元素服从 Bernoulli 分布：

Inverted Dropout 的期望一致性

设某层激活向量为 h，drop 概率为 p，保留概率为 $1-p$。训练时令 mask $m \sim \text{Bernoulli}(1-p)$，并定义输出为

$$\tilde{h} = \frac{m \odot h}{1-p}.$$

则对每个分量有

$$\mathbb{E}[\tilde{h}_i] = \frac{\mathbb{E}[m_i]h_i}{1-p} = \frac{(1-p)h_i}{1-p} = h_i.$$

这说明训练时做缩放后，推理阶段直接关闭 Dropout 即可保持期望尺度一致。

所以很多框架采用的是 **inverted dropout**：训练时先缩放，测试时直接恒等映射，不需要额外再乘保留率。这里“尺度一致”说的是输出的期望尺度保持一致，而不是说训练与测试时每次前向的具体激活值完全相同。

3 训练与推理模式

Train vs Eval：

- **训练模式（`model.train()`）**：Dropout 按设定概率随机失活部分单元，并对保留激活做缩放，从而在训练中注入噪声。
- **推理模式（`model.eval()`）**：Dropout 被关闭，层输出退化为恒等映射，模型预测保持稳定。
- **常见错误**：如果验证或测试前忘记切换到 `eval()`，Dropout 仍会随机生效，导致指标波动，且结果通常偏低。

4 超参数与调参流程

可参考的起点：

- 输入层若使用 Dropout，常见起点是 $p = 0.1 \sim 0.2$。
- 全连接隐藏层常见起点是 $p = 0.3 \sim 0.5$。
- 卷积层通常更保守，常见起点是 $p = 0.1 \sim 0.3$。
- Transformer 中常见默认值是 `dropout=0.1`，常出现在 attention、residual 和 FFN 等位置。

调参顺序参考：一般建议先固定学习率与 batch size，再调 dropout。如果训练集准确率长时间无提升，正则化可能过强，应适当减小 p；如训练集准确率远超验证集，可尝试增大 p。

一个提醒： Dropout 并不总能无条件提升测试精度。其效果与模型、数据量、轮数、数据增强和是否结合其他正则化等多因素相关，需结合具体任务实际调参。

5 PyTorch 代码对比

实验说明：下面用 CIFAR-10 上的一个小型 CNN 做简单对照实验：保持网络主体结构基本一致，只比较“不使用 Dropout”和“在全连接路径中加入 Dropout”两种设置，观察训练集与测试集指标的变化。

1) 依赖、设备与 CIFAR-10 数据加载

```python
import torch
from torch import nn, optim
from torch.nn import functional as F
import torchvision
from torchvision import transforms
from torch.utils.data import DataLoader
import wandb

# 选择可用的 GPU，如果没有则使用 CPU
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

# 构建训练和测试的数据加载器
BATCH_SIZE = 32
transform = transforms.Compose(
    [
        transforms.ToTensor(),  # 转为张量
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))  # 标准化
    ]
)

# 下载并构建训练集
trainset = torchvision.datasets.CIFAR10(
    root="./data", train=True, download=True, transform=transform
)
trainloader = DataLoader(trainset, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=2)

# 下载并构建测试集
testset = torchvision.datasets.CIFAR10(
    root="./data", train=False, download=True, transform=transform
)
testloader = DataLoader(testset, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=2)
```

2) 通用训练/评估函数（两组实验共用）

```python
def train(model, device, train_loader, optimizer, criterion, epoch):
    # 训练模式: dropout/bn 处于训练状态
    model.train()
    train_loss, train_total, train_correct = 0.0, 0, 0

    for data, target in train_loader:
        data, target = data.to(device), target.to(device)

        # 1）清除梯度
        optimizer.zero_grad()
        # 2）正向传播
        output = model(data)
        # 3）计算损失
        loss = criterion(output, target)
        train_loss += loss.item()
        # 4）计算准确率等指标
        _, predictions = torch.max(output.data, 1)
        train_total += target.size(0)
        train_correct += int(sum(predictions == target))
        # 5）反向传播和优化器更新
        loss.backward()
        optimizer.step()

    acc = round((train_correct / train_total) * 100, 2)
    print(f"Epoch [{epoch}], Loss: {train_loss/train_total}, Accuracy: {acc}", end="")
    wandb.log({"Train Loss": train_loss/train_total, "Train Accuracy": acc, "Epoch": epoch})
```

接上

```python
def test(model, device, test_loader, criterion):
    # 测试（验证）模式: dropout 自动禁用
    model.eval()
    test_loss, test_total, test_correct = 0.0, 0, 0

    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            test_loss += criterion(output, target).item()
            _, predictions = torch.max(output.data, 1)
            test_total += target.size(0)
            test_correct += int(sum(predictions == target))

    acc = round((test_correct / test_total) * 100, 2)
    print(f" Test_loss: {test_loss/test_total}, Test_accuracy: {acc}")
    wandb.log({"Test Loss": test_loss/test_total, "Test Accuracy": acc})
```

3) 基线模型：不加 Dropout

```python
class NetNoDropout(nn.Module):
    # Same CNN backbone as notebook baseline
    def __init__(self, input_shape=(3, 32, 32)):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)
        self.conv2 = nn.Conv2d(32, 64, 3)
        self.conv3 = nn.Conv2d(64, 128, 3)
        self.pool = nn.MaxPool2d(2, 2)
        n_size = self._get_conv_output(input_shape)
        self.fc1 = nn.Linear(n_size, 512)
        self.fc2 = nn.Linear(512, 10)

    def _get_conv_output(self, shape):
        sample = torch.rand(1, *shape)
        feat = self._forward_features(sample)
        return feat.view(1, -1).size(1)

    def _forward_features(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        return x

    def forward(self, x):
        x = self._forward_features(x)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

4) 对照模型：在全连接前后加入 Dropout(0.25)

```python
class NetWithDropout(nn.Module):
    # 相同的CNN骨干网络，只在全连接路径中加入Dropout
    def __init__(self, input_shape=(3, 32, 32), p=0.25):
        super().__init__()
        self.conv1 = nn.Conv2d(3, 32, 3)      # 第一层卷积
        self.conv2 = nn.Conv2d(32, 64, 3)     # 第二层卷积
        self.conv3 = nn.Conv2d(64, 128, 3)    # 第三层卷积
        self.pool = nn.MaxPool2d(2, 2)        # 最大池化层
        n_size = self._get_conv_output(input_shape)  # 计算展平后的特征数
        self.fc1 = nn.Linear(n_size, 512)     # 第一层全连接
        self.fc2 = nn.Linear(512, 10)         # 输出层（10类）
        self.dropout = nn.Dropout(p)          # Dropout层，概率p

    def _get_conv_output(self, shape):
        # 随机生成一个输入来推断展平后特征维度
        sample = torch.rand(1, *shape)
        feat = self._forward_features(sample)
        return feat.view(1, -1).size(1)

    def _forward_features(self, x):
        # 前向传播卷积和池化
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        return x

    def forward(self, x):
        # 前向过程，含Dropout
        x = self._forward_features(x)
        x = x.view(x.size(0), -1)     # 展平特征
        x = self.dropout(x)           # 第一次Dropout
        x = F.relu(self.fc1(x))       # 全连接激活
        x = self.dropout(x)           # 第二次Dropout
        x = self.fc2(x)               # 输出层
        return x
```

5) 训练入口（先跑 baseline，再跑含 Dropout 模型对照）

```python
criterion = nn.CrossEntropyLoss()  #损失函数为交叉熵
EPOCHS = 8   # 设置训练轮数为8

# Run A: baseline model (no dropout)
net_a = NetNoDropout().to(device)  # 创建不含Dropout的模型并转到指定设备
optimizer_a = optim.Adam(net_a.parameters())  # 定义优化器为Adam

# 初始化wandb记录，命名为no_dropout
wandb.init(project="dropout", name="no_dropout")

for epoch in range(EPOCHS):
    train(net_a, device, trainloader, optimizer_a, criterion, epoch)  # 训练baseline
    test(net_a, device, testloader, criterion)   # 测试
wandb.finish()  # 结束wandb记录

# Run B: dropout model
net_b = NetWithDropout(p=0.25).to(device)

optimizer_b = optim.Adam(net_b.parameters())

# 初始化wandb记录，命名为dropout_p025
wandb.init(project="dropout", name="dropout_p025")

for epoch in range(EPOCHS):
    train(net_b, device, trainloader, optimizer_b, criterion, epoch)
    test(net_b, device, testloader, criterion)
wandb.finish()
```

已跑结果 A：不加 Dropout（来自 notebook 输出）

```
Epoch [4], Loss: 0.019362751116752626, Accuracy: 78.41 Test_loss:
    0.025028224059939385, Test_accuracy: 72.85
Epoch [5], Loss: 0.016775361541509628, Accuracy: 80.84 Test_loss:
    0.025562773716449737, Test_accuracy: 73.13
Epoch [6], Loss: 0.014689673398733139, Accuracy: 83.48 Test_loss:
    0.02575759674012661, Test_accuracy: 74.17
Epoch [7], Loss: 0.012367989869192242, Accuracy: 85.83 Test_loss:
    0.026467325595021247, Test_accuracy: 73.82
```

已跑结果 B：Dropout(p=0.25)（来自 notebook 输出）

```
Epoch [4], Loss: 0.025060642061829567, Accuracy: 71.89 Test_loss:
    0.02435509083867073, Test_accuracy: 73.14
Epoch [5], Loss: 0.023284894342422487, Accuracy: 73.73 Test_loss:
    0.024100940078496932, Test_accuracy: 73.0
Epoch [6], Loss: 0.021954598691761494, Accuracy: 75.21 Test_loss:
    0.023742960724234582, Test_accuracy: 73.71
Epoch [7], Loss: 0.020816293748021127, Accuracy: 76.61 Test_loss:
    0.022519656877219678, Test_accuracy: 75.24
```

结果小结：

加入 Dropout 后训练精度下降更慢（正则化生效），但测试精度更稳，最终 epoch 的 Test_accuracy 从 73.82 提升到 75.24（+1.42）。

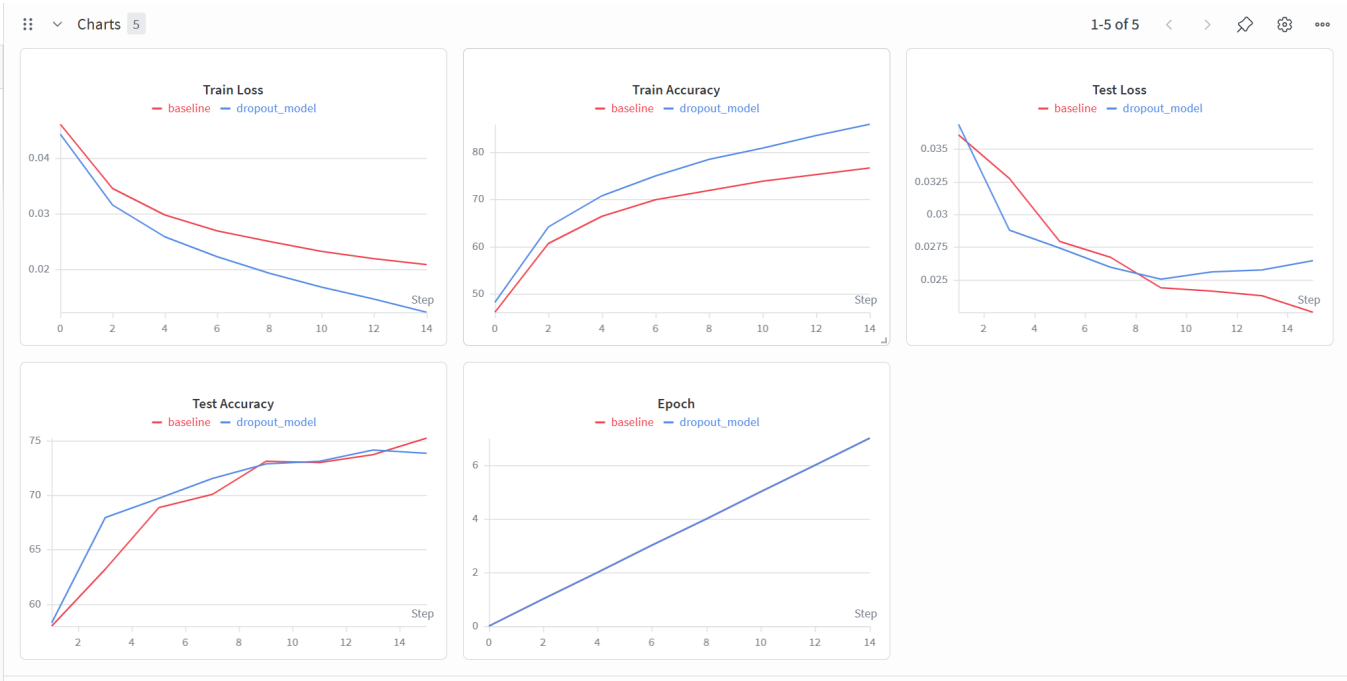

图 4: 有无 Dropout 训练与测试准确率变化对比